

1 Computational Geometry

1.1 basic

```
1 struct point {
2     double x, y;
3     point(double x = 0, double y = 0) : x(x)
4         , y(y) {}
5     point operator - (const point& b) const
6     { return point(x - b.x, y - b.y); }
7     point operator + (const point& b) const
8     { return point(x + b.x, y + b.y); }
9     point operator * (double k) const {
10         return point(x * k, y * k); }
11 };
12 double cross(point a, point b) {return a.x *
13     b.y - a.y * b.x;}
14 // 直線ab與直線cd的交點
15 point intersect(point a, point b, point c,
16     point d)
17 {
18     double c1 = cross(b - a, c - a);
19     double c2 = cross(b - a, d - a);
20     return c + (d - c) * (c1 / (c1 - c2));
21 }
```

1.2 CHT

```
1 // 當放入的直線斜率單調且查詢單調可以使用
2 deque
3 // 否則用李超樹
4 struct line
5 {
6     int m, b;
7     int val(int x){
8         return m * x + b;
9     }
10 };
11 bool bad(Line a, Line b, Line c)
12 {
13     // ab交點x座標 >= bc交點x座標
14     // ((b.b - a.b) / (a.m - b.m)) >= ((c.b - b.b) / (b.m - c.m))
15     return (b.b - a.b) * (b.m - c.m) >= (c.b - b.b) * (a.m - b.m);
16 }
17 deque<Line> dq;
18 dq.pb({x, 0});
19 for(int i = 1; i <= n; i++){
20     while(dq.size() > 1 and dq[0].val(s[i])
21         >= dq[1].val(s[i])) dq.pop_front();
22     dp[i] = dq[0].val(s[i]);
23     Line tmp = {f[i], dp[i]};
24     while(dq.size() >= 2 and bad(dq[dq.size()
25         - 2], dq.back(), tmp)) dq.
26         pop_back();
27     dq.pb(tmp);
28 }
```

1.3 convex-hull

```
1 struct node{
2     int x, y;
3     node operator - (const node &b) {
4         return {x - b.x, y - b.y};
5     }
6     int operator * (const node &b) {
7         return x * b.y - y * b.x;
8     }
9     bool operator < (const node &b) {
10         if (x != b.x) return x < b.x;
11         return y < b.y;
12     }
13 };
14 vector<node> build(vector<node> v)
15 {
16     vector<node> ans;
17     sort(v.begin(), v.end());
18     for(int i = 0; i < v.size(); i++){
19         while(ans.size() > 1 and (ans.end()
20             [-2] - ans.end()[-1]) * (ans.end()
21             )[-2] - v[i]) > 0){
22             ans.pop_back();
23         }
24         ans.pb(v[i]);
25     }
26     for(int i = v.size() - 2; i >= 0; i--){
27         while(ans.size() > 1 and (ans.end()
28             [-2] - ans.end()[-1]) * (ans.end()
29             )[-2] - v[i]) > 0){
30             ans.pop_back();
31         }
32         ans.pb(v[i]);
33     }
34     ans.pop_back();
35     return ans;
36 }
```

1.4 Line Segment Intersection

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<int, int>
7 #define F first
8 #define S second
9 #define int long long
10 pii f(pii a, pii b)
11 {
12     return {b.F - a.F, b.S - a.S};
13 }
14 int across(pii a, pii b)
15 {
16     if(a.F * b.S < a.S * b.F) return -1;
17     if(a.F * b.S > a.S * b.F) return 1;
18     return 0;
19 }
20 bool overlap(pii a, pii b, pii c, pii d)
```

```
21 {
22     if(a.F == b.F and b.F == c.F and c.F ==
23         d.F){
24         if(min(a.S, b.S) > max(c.S, d.S) or
25             min(c.S, d.S) > max(a.S, b.S))
26             return false;
27     }
28     if(min(a.F, b.F) > max(c.F, d.F) or min(
29         c.F, d.F) > max(a.F, b.F)) return
30         false;
31     return true;
32 }
33 void solve()
34 {
35     int x1, x2, x3, x4, y1, y2, y3, y4;
36     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3
37         >> x4 >> y4;
38     pii a = {x1, y1};
39     pii b = {x2, y2};
40     pii c = {x3, y3};
41     pii d = {x4, y4};
42     int x = across(f(a, b), f(a, c));
43     int y = across(f(a, b), f(a, d));
44     int z = across(f(c, d), f(c, a));
45     int w = across(f(c, d), f(c, b));
46     if(x * y < 0 and z * w < 0) cout << "YES
47         \n";
48     else if(x * y <= 0 and z * w <= 0 and
49         overlap(a, b, c, d))cout << "YES\n";
50     else cout << "NO\n";
51 }
52 signed main()
53 {
54     KCC
55     int t;
56     cin >> t;
57     while(t--){ solve();
58         return 0;
59     }
```

1.5 Point in Polygon

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 int x[(int)1e3 + 5], y[(int) 1e3 + 5], n, m;
11 bool onsegment(pii a, pii b, pii p)
12 {
13     if((a.F - b.F) * (a.S - p.S) != (a.F - p
14         .F) * (a.S - b.S)) return false;
15     if(!(min(a.F, b.F) <= p.F and p.F <= max
16         (a.F, b.F))) return false;
17     if(!(min(a.S, b.S) <= p.S and p.S <= max
18         (a.S, b.S))) return false;
19     return true;
20 }
```

```
18 }
19 signed main()
20 {
21     KCC
22     cin >> n >> m;
23     for(int i = 0; i < n; i++) cin >> x[i]
24         >> y[i];
25     while(m--){
26         int a, b;
27         cin >> a >> b;
28         pii p = {a, b};
29         bool boundary = false;
30         int cnt = 0;
31         for(int i = 0; i < n; i++){
32             pii u = {x[i], y[i]}, v = {x[(i
33                 + 1) % n], y[(i + 1) % n]};
34             if(onsegment(u, v, p)){
35                 boundary = true;
36                 break;
37             }
38             if(min(u.S, v.S) <= b and b <
39                 max(u.S, v.S)){
40                 /*
41                 Line u_to_v:
42                 X = u.F + t(v.F - u.F)
43                 Y = u.S + t(v.S - u.S)
44                 b = u.S + t(v.S - u.S)
45                 t = (b - u.S) / (v.S - u.S)
46                 X = u.F + (b - u.S) / (v.S -
47                     u.S) * (v.F - u.F)
48                 */
49                 double X = u.F + 1.0 * (b -
50                     u.S) / (v.S - u.S) * (v.
51                         F - u.F);
52                 if(a <= X) cnt++;
53             }
54         }
55         if (boundary) cout << "BOUNDARY\n";
56         else if (cnt & 1) cout << "INSIDE\n";
57         else cout << "OUTSIDE\n";
58     }
```

1.6 半平面交集

```
1 // 用有向線段 AB 切割多邊形 v，保留左半邊
2 vector<point> cut(vector<point> v, point a,
3     point b)
4 {
5     vector<point> ans;
6     int n = v.size();
7     for(int i = 0; i < n; i++){
8         point c = v[i], d = v[(i + 1) % n];
9         double side_c = cross(b - a, c - a);
10        double side_d = cross(b - a, d - a);
11        if(side_c >= 0){
12            ans.pb(c);
13            if(side_d < 0) ans.pb(intersect(
14                a, b, c, d));
15        }
```

```

15     }
16     else{
17         if(side_d >= 0) ans.pb(intersect
18             (a, b, c, d));
19     }
20     return ans;
21 }

```

1.7 最近點對

```

1 set<pii> s; // {y, x}
2 for(int i = 0; i < n; i++){
3     int d = sqrt(ans) + 1;
4     while(s.size() > 0 and arr[i].F - arr[id
5         ].F >= d){
6         s.erase({arr[id].S, arr[id].F});
7         id++;
8     }
9     auto l = s.lower_bound({arr[i].S - d, -1
10         e9});
11     auto r = s.lower_bound({arr[i].S + d, 1
12         e9});
13     for(auto j = l; j != r; ++j) ans = min(
14         ans, dis(i, *j));
15     s.insert({arr[i].S, arr[i].F});
16 }

```

1.8 李超線段樹

```

1 struct Line{
2     int k, b;
3     int val(int x){
4         return k * x + b;
5     }
6 };
7 Line tree[(int)1e6]; //維護最大值
8 bool has_line[(int)1e6];
9 void insert(int node, int l, int r, Line
10     new_line)
11 {
12     if(!has_line[node]){
13         tree[node] = new_line;
14         has_line[node] = true;
15         return;
16     }
17     int mid = (l + r) >> 1;
18     bool l_better = new_line.val(l) > tree[
19         node].val(l);
20     bool mid_better = new_line.val(mid) >
21         tree[node].val(mid);
22     if(mid_better) swap(new_line, tree[node
23         ]);
24     if(l == r) return;
25     if(l_better != mid_better) insert(node
26         << 1, l, mid, new_line);
27     else insert(node << 1 | 1, mid + 1, r,
28         new_line);
29 }

```

```

24 int query(int node, int l, int r, int x)
25 {
26     if(!has_line[node]) return -2e18;
27     int re = tree[node].val(x);
28     if(l == r) return re;
29     int mid = (l + r) >> 1;
30     if(x <= mid) re = max(re, query(node <<
31         1, l, mid, x));
32     else re = max(re, query(node << 1 | 1,
33         mid + 1, r, x));
34     return re;
35 }

```

1.9 矩形覆蓋面積

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4     (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 struct SEG{
11     int cnt, len;
12 }seg[(int)1e7];
13 void pull(int id, int l, int r)
14 {
15     if(seg[id].cnt > 0) seg[id].len = r - l
16         + 1;
17     else{
18         if(l != r) seg[id].len = seg[id <<
19             1].len + seg[id << 1 | 1].len;
20         else seg[id].len = 0;
21     }
22 }
23 void update(int ql, int qr, int x, int l,
24     int r, int id)
25 {
26     if(ql <= l and r <= qr){
27         seg[id].cnt += x;
28         pull(id, l, r);
29         return;
30     }
31     int mid = (l + r) >> 1;
32     if(ql <= mid) update(ql, qr, x, l, mid,
33         id << 1);
34     if(qr > mid) update(ql, qr, x, mid + 1,
35         r, id << 1 | 1);
36     pull(id, l, r);
37 }
38 signed main()
39 {
40     KCC
41     int n, ans = 0;
42     cin >> n;
43     vector<array<int, 4>> v;
44     for(int i = 0; i < n; i++){
45         int a, b, c, d;
46         cin >> a >> b >> c >> d;
47         a += 1e6 + 1;

```

```

43     b += 1e6 + 1;
44     c += 1e6 + 1;
45     d += 1e6 + 1;
46     v.pb({a, b, d, 1});
47     v.pb({c, b, d, -1});
48 }
49 sort(v.begin(), v.end());
50 int last = v[0][0], maxval = 2e6 + 1;
51 for(auto [x, y1, y2, sign] : v){
52     ans += (x - last) * seg[1].len;
53     update(y1, y2 - 1, sign, 1, maxval,
54         1);
55     last = x;
56 }
57 cout << ans << "\n";
58 return 0;

```

2 Data Structure

2.1 DSU-rollback

```

1 struct DSU_rollback{
2     int components;
3     vector<int> parent, sz;
4     struct modify{
5         int u, v, pre_sz;
6     };
7     vector<modify> history;
8     void initial(int n){
9         parent.resize(n + 1);
10        sz.resize(n + 1, 1);
11        components = n;
12        for(int i = 1; i <= n; i++) parent[i
13            ] = i;
14    }
15    int find(int x){
16        while(x != parent[x]) x = parent[x];
17        return x;
18    }
19    void unite(int u, int v){
20        int root_u = find(u), root_v = find(
21            v);
22        if(root_u == root_v) return;
23        if(sz[root_u] < sz[root_v]) swap(
24            root_u, root_v);
25        history.pb({root_u, root_v, sz[
26            root_u]});
27        parent[root_v] = root_u;
28        sz[root_u] += sz[root_v];
29        components--;
30    }
31    void rollback(){
32        if(history.size() == 0) return;
33        modify last = history.back();
34        history.pop_back();
35        parent[last.v] = last.v;
36        sz[last.u] = last.pre_sz;
37        components++;
38    }
39 }

```

```

35 int dsuetime(){return (int)history.size()
36     ;}
37 void return_to_t(int t){
38     while(history.size() > t) rollback()
39     ;}

```

2.2 FHQ-treap

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4     (0);
5 #define pb push_back
6 #define pii pair<int, int>
7 #define F first
8 #define S second
9 #define mt19937 rnd(time(0));
10 const int MAXN = 2e5 + 5;
11 int left_node[MAXN], right_node[MAXN],
12     priority[MAXN], sz[MAXN], node_cnt = 0,
13     root = 0;
14 char val[MAXN];
15 void up(int i)
16 {
17     sz[i] = sz[left_node[i]] + sz[right_node
18         [i]] + 1;
19 }
20 void split(int i, int l, int r, int rank)
21 {
22     if(i == 0){
23         left_node[r] = right_node[l] = 0;
24         return;
25     }
26     if(sz[left_node[i]] + 1 <= rank){
27         right_node[l] = i;
28         split(right_node[i], i, r, rank - sz
29             [left_node[i]] - 1);
30     }
31     else{
32         left_node[r] = i;
33         split(left_node[i], l, i, rank);
34     }
35     up(i);
36 }
37 int merge(int l, int r)
38 {
39     if(l == 0 or r == 0) return l + r;
40     if(priority[l] >= priority[r]){
41         right_node[l] = merge(right_node[l],
42             r);
43         up(l);
44         return l;
45     }
46     else{
47         left_node[r] = merge(l, left_node[r
48             ]);
49         up(r);
50         return r;
51     }
52 }
53 void out(int i)

```

```

47 {
48     if(i == 0) return;
49     out(left_node[i]);
50     cout << val[i];
51     out(right_node[i]);
52 }
53 int main()
54 {
55     KCC
56     int n, m;
57     cin >> n >> m;
58     for(int i = 0; i < n; i++){
59         char c;
60         cin >> c;
61         priority[++node_cnt] = rnd();
62         val[node_cnt] = c;
63         sz[node_cnt] = 1;
64         root = merge(root, node_cnt);
65     }
66     while(m--){
67         int a, b;
68         cin >> a >> b;
69         split(root, 0, 0, b);
70         int lm = right_node[0];
71         int r = left_node[0];
72         split(lm, 0, 0, a - 1);
73         int l = right_node[0];
74         int m = left_node[0];
75         root = merge(merge(l, r), m);
76     }
77     out(root);
78     return 0;
79 }

```

2.3 persistent-segment-tree

```

1 int upd(int pre, int l, int r, int pos, int
  val)
2 {
3     int node = ++cnt_tree;
4     tree[node] = tree[pre];
5     if(l == r){
6         tree[node].sum += val;
7         return node;
8     }
9     int mid = (l + r) >> 1;
10    if(pos <= mid) tree[node].l = upd(tree[
    pre].l, l, mid, pos, val);
11    else tree[node].r = upd(tree[pre].r, mid
    + 1, r, pos, val);
12    tree[node].sum = tree[tree[node].l].sum
    + tree[tree[node].r].sum;
13    return node;
14 }
15 int query(int node, int l, int r, int ql,
    int qr)
16 {
17     if(ql <= l and r <= qr) return tree[node
    ].sum;
18     int mid = (l + r) >> 1, re = 0;
19     if(ql <= mid) re += query(tree[node].l,
    l, mid, ql, qr);

```

```

20     if(qr > mid) re += query(tree[node].r,
    mid + 1, r, ql, qr);
21     return re;
22 }
23 int get_kth(int node_l, int node_r, int l,
    int r, int k)
24 {
25     //原陣列位置區間(node_l, node_r)第k小
26     /*
27     if(l == r) return l;
28     int mid = (l + r) >> 1;
29     int left_cnt = tree[tree[node_r].l].sum
    - tree[tree[node_l].l].sum;
30     if(k <= left_cnt) return get_kth(tree[
    node_l].l, tree[node_r].l, l, mid, k
    );
31     else return get_kth(tree[node_l].r, tree
    [node_r].r, mid + 1, r, k - left_cnt
    );
32     */
33     while (l < r) {
34         int mid = (l + r) >> 1;
35         int left_cnt = tree[tree[node_r].l].
    sum - tree[tree[node_l].l].sum;
36         if (k <= left_cnt) {
37             node_l = tree[node_l].l;
38             node_r = tree[node_r].l;
39             r = mid;
40         } else {
41             k -= left_cnt;
42             node_l = tree[node_l].r;
43             node_r = tree[node_r].r;
44             l = mid + 1;
45         }
46     }
47     return l;
48 }

```

2.4 segment tree lazytag

```

1 struct SegmentTree{
2     struct Node{
3         int sum, f, d, len;
4     } tree[MAXN << 2];
5     void pull(int id){
6         tree[id].sum = tree[id << 1].sum +
    tree[id << 1 | 1].sum;
7     }
8     void addtag(int id, int a, int d){
9         tree[id].f += a;
10        tree[id].d += d;
11        tree[id].sum += (2 * a + (tree[id].
    len - 1) * d) * tree[id].len /
    2;
12    }
13    void push(int id){
14        if(!tree[id].f && !tree[id].d)
    return;
15        int mid_f = tree[id].f + tree[id <<
    1].len * tree[id].d;
16        addtag(id << 1, tree[id].f, tree[id
    ].d);

```

```

17        addtag(id << 1 | 1, mid_f, tree[id].
    d);
18        tree[id].f = tree[id].d = 0;
19    }
20    void build(int l, int r, int id, int ar
    []){
21        tree[id].len = r - l + 1;
22        tree[id].f = tree[id].d = 0;
23        if(l == r){
24            tree[id].sum = ar[l];
25            return;
26        }
27        int mid = (l + r) >> 1;
28        build(l, mid, id << 1, ar);
29        build(mid + 1, r, id << 1 | 1, ar);
30        pull(id);
31    }
32    void update(int ql, int qr, int a, int d
    , int l, int r, int id){
33        if(ql <= l and r <= qr){
34            addtag(id, a + (l - ql) * d, d);
35            return;
36        }
37        push(id);
38        int mid = (l + r) >> 1;
39        if(ql <= mid) update(ql, qr, a, d, l
    , mid, id << 1);
40        if(qr > mid) update(ql, qr, a, d,
    mid + 1, r, id << 1 | 1);
41        pull(id);
42    }
43    int query(int ql, int qr, int l, int r,
    int id){
44        if(ql <= l and r <= qr) return tree[
    id].sum;
45        push(id);
46        int mid = (l + r) >> 1, re = 0;
47        if(ql <= mid) re += query(ql, qr, l,
    mid, id << 1);
48        if(qr > mid) re += query(ql, qr, mid
    + 1, r, id << 1 | 1);
49        return re;
50    }
51 }

```

2.5 trie

```

1 struct Node{
2     int next_node[2]; // 01-trie
3     int max_idx;
4     Node() {
5         next_node[0] = next_node[1] = -1;
6         max_idx = -1;
7     }
8 };
9 vector<Node> trie;
10 void init_trie()
11 {
12     trie.clear();
13     trie.emplace_back(); // 插入根節點
14 }
15 void insert(int x, int id)
16 {

```

```

17     int u = 0;
18     for(int i = 29; i >= 0; i--){
19         int bit = (x >> i) & 1;
20         if(trie[u].next_node[bit] == -1){
21             trie[u].next_node[bit] = trie.
    size();
22             trie.emplace_back();
23         }
24         u = trie[u].next_node[bit];
25         trie[u].max_idx = max(trie[u].
    max_idx, id);
26     }
27 }
28 // 減少空間的存法
29 struct Edge {
30     int to, nxt;
31     char c;
32 };
33 struct Node {
34     int head = -1;
35     // 放節點要存的資訊
36 };
37 vector<Node> trie;
38 vector<Edge> edges;
39 int get_child(int u, char c){
40     for(int e = trie[u].head; e != -1; e =
    edges[e].nxt){
41         if(edges[e].c == c) return edges[e].
    to;
42     }
43     return -1;
44 }
45 int nxt_child(int u, char c){
46     int v = get_child(u, c);
47     if(v != -1) return v;
48
49     v = trie.size();
50     trie.emplace_back();
51
52     edges.push_back({v, trie[u].head, c});
53     trie[u].head = (int)edges.size() - 1;
54     return v;
55 }
56 }

```

2.6 Weighted DSU

```

1 struct Weighted_DSU{
2     vector<int> parent, dis;
3     void initial(int n){
4         parent.resize(n + 1);
5         dis.resize(n + 1, 0);
6         for(int i = 1; i <= n; i++) parent[i
    ] = i;
7     }
8     int find(int x){
9         if(parent[x] != x){
10             int tmp = parent[x];
11             parent[x] = find(tmp);
12             dis[x] += dis[tmp];
13         }
14         return parent[x];

```

```

15 }
16 void unite(int l, int r, int d){
17     int lf = find(l), rf = find(r);
18     if(lf != rf){
19         parent[lf] = rf;
20         dis[lf] = dis[r] - dis[l] + d;
21     }
22 }
23 int query(int l, int r){
24     if(find(l) != find(r)) return 1e18;
25     return dis[l] - dis[r];
26 }
27 };

```

2.7 動態開點線段樹

```

1 struct Node{
2     int sum = 0;
3     Node *l = nullptr;
4     Node *r = nullptr;
5 }*root = nullptr;
6
7 void add(Node *&node, int L, int R, int pos,
8         int val)
9 {
10     if(!node) node = new Node();
11     node->sum += val;
12     if(L == R) return;
13     int mid = (L + R) / 2;
14     if(pos <= mid) add(node->l, L, mid, pos,
15         val);
16     else add(node->r, mid + 1, R, pos, val);
17 }
18
19 int query(Node *node, int L, int R, int ql,
20         int qr)
21 {
22     if(!node) return 0;
23     if(ql <= L and R <= qr) return node->sum;
24     ;
25     int mid = (L + R) / 2, ans = 0;
26     if(ql <= mid) ans += query(node->l, L,
27         mid, ql, qr);
28     if(qr > mid) ans += query(node->r, mid +
29         1, R, ql, qr);
30     return ans;
31 }

```

3 Flow

3.1 dinic

```

1 struct Dinic{
2     struct Edge{
3         int to, cap, rev;
4     };
5     vector<vector<Edge>> adj;
6     vector<int> level, ptr;
7     int n;

```

```

8     Dinic(int _n){
9         n = _n;
10        level.resize(n + 1);
11        ptr.resize(n + 1);
12        adj.resize(n + 1);
13    }
14    void addedge(int u, int v, int w){
15        adj[u].pb({v, w, adj[v].size()});
16        adj[v].pb({u, 0, adj[u].size() - 1});
17    }
18    bool bfs(int s, int t)
19    {
20        fill(level.begin(), level.end(), -1)
21        ;
22        level[s] = 0;
23        queue<int> q;
24        q.push(s);
25        while(q.size()){
26            int x = q.front(); q.pop();
27            for(auto edge : adj[x]){
28                if(edge.cap > 0 and level[
29                    edge.to] == -1){
30                    level[edge.to] = level[x]
31                        + 1;
32                    q.push(edge.to);
33                }
34            }
35        }
36        return (level[t] != -1);
37    }
38    int dfs(int now, int t, int pushed)
39    {
40        if(now == t) return pushed;
41        for(int &i = ptr[now]; i < adj[now].
42            size(); i++){
43            auto &w = adj[now][i];
44            if(level[now] + 1 != level[w.to]
45                or w.cap == 0) continue;
46            int tr = dfs(w.to, t, min(pushed
47                , w.cap));
48            if(tr == 0) continue;
49            w.cap -= tr;
50            adj[w.to][w.rev].cap += tr;
51            return tr;
52        }
53        return 0;
54    }
55    int compute_max_flow(){
56        int max_flow = 0;
57        while(bfs(1, n)){
58            fill(ptr.begin(), ptr.end(), 0);
59            while(int pushed = dfs(1, n, 1
60                e18)){
61                max_flow += pushed;
62            }
63        }
64        return max_flow;
65    }
66 };

```

3.2 Edmonds-Karp

```

1 struct max_flow{
2     int n;
3     vector<int> dis, par, mn;
4     vector<vector<int>> g;
5     vector<array<int, 3>> edge;
6     max_flow(int _n){
7         n = _n;
8         dis.resize(n + 5);
9         par.resize(n + 5);
10        mn.resize(n + 5);
11        g.resize(n + 5);
12    }
13    void add_edge(int a, int b, int c){
14        g[a].pb(edge.size());
15        edge.pb({a, b, c});
16        g[b].pb(edge.size());
17        edge.pb({b, a, 0});
18    }
19    int bfs()
20    {
21        for(int i = 1; i <= n; i++) dis[i] =
22            -1;
23        for(int i = 1; i <= n; i++) mn[i] =
24            1e18;
25        dis[1] = 0;
26        queue<int> q;
27        q.push(1);
28        while(q.size()){
29            int x = q.front(); q.pop();
30            for(int i : g[x]){
31                auto [now, to, w] = edge[i];
32                if(dis[to] == -1 and w > 0){
33                    dis[to] = dis[now] + 1;
34                    par[to] = i;
35                    mn[to] = min(mn[now], w)
36                        ;
37                    q.push(to);
38                }
39            }
40        }
41        if(dis[n] == -1) return 0;
42        int now = n;
43        while(now != 1){
44            auto &[a, b, w] = edge[par[now]
45                ];
46            auto &[ra, rb, rw] = edge[par[
47                now] ^ 1];
48            w -= mn[n];
49            rw += mn[n];
50            now = a;
51        }
52        return mn[n];
53    }
54    int max_flow_ans(){
55        int ans = 0;
56        while(true){
57            int t = bfs();
58            if(t == 0) break;
59            ans += t;
60        }
61        return ans;
62    }
63 };

```

3.3 Min Cost Max-Flow

```

1 struct Edge {
2     ll to, cap, cost, rev;
3     Edge() {}
4     Edge(int _to, ll _cap, ll _cost, int
5         _rev) : to(_to), cap(_cap), cost(
6             _cost), rev(_rev) {}
7 };
8 vector<Edge> adj[N];
9 ll dis[N], ll par[N], par_id[N]; bool
10 in_que[N];
11 pair<ll, ll> MCMF(int s, int t) {
12     ll flow = 0, cost = 0;
13     while(true) {
14         memset(dis, 0x3f, sizeof(dis));
15         memset(in_que, 0, sizeof(in_que));
16         queue<int> que;
17         que.push(s); dis[s] = 0;
18         while(!que.empty()) { // SPFA
19             int x = que.front();
20             que.pop(); in_que[x] = 0;
21             for(int i = 0; i < (int)adj[x].
22                 size(); i++) {
23                 Edge &e = adj[x][i];
24                 if(e.cap > 0 && dis[e.to] >
25                     dis[x] + e.cost) {
26                     dis[e.to] = dis[x] + e.
27                         cost;
28                     par[e.to] = x;
29                     par_id[e.to] = i;
30                     if(!in_que[e.to]) {
31                         in_que[e.to] = 1;
32                         que.push(e.to);
33                     }
34                 }
35             }
36         }
37         if(dis[t] >= INF) break;
38         ll mi_flow = INF;
39         for(int i = t; i != s; i = par[i])
40             mi_flow = min(mi_flow, adj[par[i]
41                 ][par_id[i]].cap);
42         flow += mi_flow, cost += mi_flow *
43             dis[t];
44         for(int i = t; i != s; i = par[i]) {
45             Edge &e = adj[par[i]][par_id[i]
46                 ];
47             e.cap -= mi_flow;
48             adj[e.to][e.rev].cap += mi_flow;
49         }
50     }
51     return make_pair(flow, cost);
52 }
53 void add_edge(int a, int b, ll cap, ll cost)
54 {
55     adj[a].push_back(Edge(b, cap, cost, (int)
56         adj[b].size()));
57     adj[b].push_back(Edge(a, 0, -cost, (int)
58         adj[a].size() - 1));
59 }
60 ll hungarian(vector<vector<ll>> cost) {
61     int n = (int)cost.size();
62     for(int i = 0; i < n; i++) {
63         for(int j = 0; j < n; j++) {

```

```

52     int L = i, T = j + n;
53     add_edge(L, T, 1, cost[i][j]);
54 }
55 }
56 int s = 2 * n, t = 2 * n + 1;
57 for(int i = 0; i < n; i++) {
58     int L = i, T = i + n;
59     add_edge(s, L, 1, 0);
60     add_edge(T, t, 1, 0);
61 }
62 // MCMF(s, t)
63 for(int i = 0; i < n; i++) {
64     for(Edge &e : adj[i]) {
65         if(e.to == s) continue;
66         int row = i, col = e.to - n;
67         if(e.cap == 0)
68             // (row + 1) match (col + 1)
69     }
70 }
71 return MCMF(s, t).second;
72 }

```

4 Graph

4.1 bellman-ford

```

1 void bellman_ford(){
2     for(int i = 1; i <= n; i++){
3         bool update = false;
4         for(int j = 1; j <= n; j++){
5             for(auto k : g[j]){
6                 if(dis[k.F] > dis[j] + k.S)
7                     update = true, dis[k.F]
8                     = dis[j] + k.S;
9             if(!update)break;
10            if(i == n)cout << "neg_cycle" << "\n";
11        }
12    }
13 }

```

4.2 dijk

```

1 priority_queue<pii, vector<pii>, greater<pii>
2 >> q;
3 q.push({0, 1});
4 dis[1] = 0;
5 while(q.size()){
6     auto[d, x] = q.top(); q.pop();
7     if(d > dis[x]) continue;
8     for(auto [to, w] : g[x]){
9         if(dis[to] > dis[x] + w){
10             dis[to] = dis[x] + w;
11             q.push({dis[to], to});
12         }
13     }
14 }

```

4.3 Euler-Path

```

1 void dfs(int x){
2     while(g[x].size()){
3         int nxt = g[x].back();
4         g[x].pop_back();
5         dfs(nxt);
6     }
7     ans.pb(x); // 倒序記錄答案
8 }

```

4.4 SCC

```

1 struct SCC{
2     vector<vector<int>> g, rg;
3     vector<bool> vis, used;
4     vector<int> component, st;
5     int component_cnt = 0, n;
6     void dfs1(int node){
7         vis[node] = true;
8         for(int i : g[node]) if(!vis[i])
9             dfs1(i);
10        st.pb(node);
11    }
12    void dfs2(int node){
13        used[node] = true;
14        for(int i : rg[node]) if(!used[i])
15            dfs2(i);
16        component[node] = component_cnt;
17    }
18    SCC(const vector<vector<int>> v){
19        n = v.size() - 1;
20        g = v;
21        rg.resize(n + 1);
22        vis.resize(n + 1, false);
23        used.resize(n + 1, false);
24        component.resize(n + 1, 0);
25        for(int i = 1; i <= n; i++) for(int
26            j : v[i]) rg[j].pb(i);
27    };
28    vector<int> find_component(){
29        for(int i = 1; i <= n; i++) if(!vis[
30            i]) dfs1(i);
31        reverse(st.begin(), st.end());
32        for(int i : st) if(!used[i]){
33            component_cnt++;
34            dfs2(i);
35        }
36        return component;
37    }
38 };

```

4.5 tarjan scc

```

1 struct tarjan_scc{
2     int n, dfn_cnt = 0, scc_cnt = 0;
3     vector<int> belong, dfn, low, stack;
4     vector<vector<int>> g;
5     void initial(int _n){

```

```

6         n = _n;
7         belong.resize(n + 1, 0);
8         dfn.resize(n + 1, 0);
9         low.resize(n + 1, 0);
10        g.resize(n + 1, vector<int>(0));
11    }
12    void addedge(int u, int v){
13        g[u].pb(v);
14    }
15    void tarjan(int u){
16        dfn[u] = low[u] = ++dfn_cnt;
17        stack.pb(u);
18        for(int i : g[u]){
19            if(dfn[i] == 0){
20                tarjan(i);
21                low[u] = min(low[u], low[i])
22                ;
23            }
24            else if(belong[i] == 0){
25                low[u] = min(low[u], dfn[i])
26                ;
27            }
28        }
29        if(dfn[u] == low[u]){
30            scc_cnt++;
31            int x;
32            do{
33                x = stack.back();
34                belong[x] = scc_cnt;
35                stack.pop_back();
36            }while(x != u);
37        }
38    }
39    void work(){
40        for(int i = 1; i <= n; i++){
41            if(belong[i] == 0){
42                tarjan(i);
43            }
44        }
45    }
46 };

```

5 Linear Programming

5.1 simplex

```

1 /*target:
2     max \sum_{j=1}^n A_{0,j} * x_j
3 condition:
4     \sum_{j=1}^n A_{i,j} * x_j <= A_{i,0} | i=1~m
5     x_j >= 0 | j=1~n
6 VDB = vector<double>*/
7 template<class VDB>
8 VDB simplex(int m, int n, vector<VDB> a){
9     vector<int> left(m+1), up(n+1);
10    iota(left.begin(), left.end(), n);
11    iota(up.begin(), up.end(), 0);
12    auto pivot = [&](int x, int y){
13        swap(left[x], up[y]);
14        auto k = a[x][y]; a[x][y] = 1;
15        vector<int> pos;

```

```

16        for(int j = 0; j <= n; ++j){
17            a[x][j] /= k;
18            if(a[x][j] != 0) pos.push_back(j);
19        }
20        for(int i = 0; i <= m; ++i){
21            if(a[i][y]==0 || i == x) continue;
22            k = a[i][y], a[i][y] = 0;
23            for(int j : pos) a[i][j] -= k*a[x][j];
24        }
25    };
26    for(int x,y;;){
27        for(int i=x+1; i <= m; ++i)
28            if(a[i][0]<a[x][0]) x = i;
29        if(a[x][0]>=0) break;
30        for(int j=y+1; j <= n; ++j)
31            if(a[x][j]<a[x][y]) y = j;
32        if(a[x][y]>=0) return VDB(); //infeasible
33        pivot(x, y);
34    }
35    for(int x,y;;){
36        for(int j=y+1; j <= n; ++j)
37            if(a[0][j] > a[0][y]) y = j;
38        if(a[0][y]<=0) break;
39        x = -1;
40        for(int i=1; i<=m; ++i) if(a[i][y] > 0)
41            if(x == -1 || a[i][0]/a[i][y]
42                < a[x][0]/a[x][y]) x = i;
43        if(x == -1) return VDB(); //unbounded
44        pivot(x, y);
45    }
46    VDB ans(n + 1);
47    for(int i = 1; i <= m; ++i)
48        if(left[i] <= n) ans[left[i]] = a[i][0];
49    ans[0] = -a[0][0];
50    return ans;
51 }

```

6 Number Theory

6.1 basic

```

1 template<typename T>
2 void gcd(const T &a, const T &b, T &d, T &x, T &
3 y){
4     if(!b) d=a, x=1, y=0;
5     else gcd(b, a%b, d, y, x), y-=x*(a/b);
6 }
7 long long int phi[N+1];
8 void phiTable(){
9     for(int i=1; i<=N; i++) phi[i]=i;
10    for(int i=1; i<=N; i++) for(x=i*2; x<=N; x+=i)
11        phi[x]-=phi[i];
12 }
13 void all_divdown(const LL &n) { // all n/x
14     for(LL a=1; a<=n; a=n/(n/(a+1))) {
15         // dosomething;
16     }
17 }
18 const int MAXPRIME = 1000000;
19 int iscom[MAXPRIME], prime[MAXPRIME],
20 primecnt;

```



```

18 int phi[MAXPRIME], mu[MAXPRIME];
19 void sieve(void){
20     memset(iscom,0,sizeof(iscom));
21     primecnt = 0;
22     phi[1] = mu[1] = 1;
23     for(int i=2;i<MAXPRIME;++i) {
24         if(!iscom[i]) {
25             prime[primecnt++] = i;
26             mu[i] = -1;
27             phi[i] = i-1;
28         }
29         for(int j=0;j<primecnt;++j) {
30             int k = i * prime[j];
31             if(k>=MAXPRIME) break;
32             iscom[k] = prime[j];
33             if(i%prime[j]==0) {
34                 mu[k] = 0;
35                 phi[k] = phi[i] * prime[j];
36                 break;
37             } else {
38                 mu[k] = -mu[i];
39                 phi[k] = phi[i] * (prime[j]-1);
40             }
41         }
42     }
43 }
44
45 bool g_test(const LL &g, const LL &p, const
vector<LL> &v) {
46     for(int i=0;i<v.size();++i)
47         if(modexp(g,(p-1)/v[i],p)==1)
48             return false;
49     return true;
50 }
51 LL primitive_root(const LL &p) {
52     if(p==2) return 1;
53     vector<LL> v;
54     Factor(p-1,v);
55     v.erase(unique(v.begin(), v.end()), v.end
());
56     for(LL g=2;g<p;++g)
57         if(g_test(g,p,v))
58             return g;
59     puts("primitive_root NOT FOUND");
60     return -1;
61 }
62 int Legendre(const LL &a, const LL &p) {
63     return modexp(a%p,(p-1)/2,p); }
64
65 LL inv(const LL &a, const LL &n) {
66     LL d,x,y;
67     gcd(a,n,d,x,y);
68     return d==1 ? (x%n)%n : -1;
69 }
70
71 int inv[maxN];
72 LL invtable(int n,LL P){
73     inv[1]=1;
74     for(int i=2;i<n;++i)
75         inv[i]=(P-(P/i))*inv[P%i]%P;
76 }
77
78 LL log_mod(const LL &a, const LL &b, const
LL &p) {
79     // a ^ x = b ( mod p )
80     int m=sqrt(p+.5), e=1;

```

```

81     LL v=inv(modexp(a,m,p), p);
82     map<LL,int> x;
83     x[1]=0;
84     for(int i=1;i<m;++i) {
85         e = LLmul(e,a,p);
86         if(!x.count(e)) x[e] = i;
87     }
88     for(int i=0;i<m;++i) {
89         if(x.count(b)) return i*m + x[b];
90         b = LLmul(b,v,p);
91     }
92     return -1;
93 }
94
95 LL Tonelli_Shanks(const LL &n, const LL &p)
{
96     // x^2 = n ( mod p )
97     if(n==0) return 0;
98     if(Legendre(n,p)!=1) while(1) { puts("SQRT
ROOT does not exist"); }
99     int S = 0;
100     LL Q = p-1;
101     while( !(Q&1) ) { Q>>=1; ++S; }
102     if(S==1) return modexp(n%p,(p+1)/4,p);
103     LL z = 2;
104     for(;Legendre(z,p)!=-1;++z)
105         LL c = modexp(z,Q,p);
106         LL R = modexp(n%p,(Q+1)/2,p), t = modexp(n
%p,Q,p);
107         int M = S;
108         while(1) {
109             if(t==1) return R;
110             LL b = modexp(c,1L<<(M-i-1),p);
111             R = LLmul(R,b,p);
112             t = LLmul(LLmul(b,b,p), t, p);
113             c = LLmul(b,b,p);
114             M = i;
115         }
116     return -1;
117 }
118
119 template<typename T>
120 T Euler(T n){
121     T ans=n;
122     for(T i=2;i*i<=n;++i){
123         if(n%i==0){
124             ans=ans/i*(i-1);
125             while(n%i==0)n/=i;
126         }
127     }
128     if(n>1)ans=ans/n*(n-1);
129     return ans;
130 }
131
132 //Chinese_remainder_theorem
133 template<typename T>
134 T pow_mod(T n,T k,T m){
135     T ans=1;
136     for(n=(n>m?n%m:n);k>>=1){
137         if(k&1)ans=ans*n%m;
138         n=n*n%m;
139     }
140     return ans;
141 }
142
143 template<typename T>
144 T crt(vector<T> &m,vector<T> &a){

```

```

145     T M=1,tM,ans=0;
146     for(int i=0;i<(int)m.size();++i)M*=m[i];
147     for(int i=0;i<(int)a.size();++i){
148         tM=M/m[i];
149         ans=(ans+(a[i]*tM%M)*pow_mod(tM,Euler(m[
i])-1,m[i])%M)%M;
150         /*如果m[i]是質數 Euler(m[i])-1=m[i]-2
151         就不用算Euler了*/
152     }
153     return ans;
154 }
155
156 //java code
157 //求sqrt(N)的連分數
158 public static void Pell(int n){
159     BigInteger N,p1,p2,q1,q2,a0,a1,a2,g1,g2,h1
,h2,p,q;
160     g1=q2=p1=BigInteger.ZERO;
161     h1=q1=p2=BigInteger.ONE;
162     a0=a1=BigInteger.valueOf((int)Math.sqrt
(1.0*n));
163     BigInteger ans=a0.multiply(a0);
164     if(ans.equals(BigInteger.valueOf(n))){
165         System.out.println("No solution!");
166         return ;
167     }
168     while(true){
169         g2=a1.multiply(h1).subtract(g1);
170         h2=N.subtract(g2.pow(2)).divide(h1);
171         a2=g2.add(a0).divide(h2);
172         p=a1.multiply(p2).add(p1);
173         q=a1.multiply(q2).add(q1);
174         if(p.pow(2).subtract(N.multiply(q.pow
(2))).compareTo(BigInteger.ONE)==0)
175             break;
176         g1=g2;h1=h2;a1=a2;
177         p1=p2;p2=p;
178         q1=q2;q2=q;
179     }
180     System.out.println(p+" "+q);
181 }

```

6.2 bit set

```

1 void sub_set(int S){
2     int sub=S;
3     do{
4         //對某集合的子集的處理
5         sub=(sub-1)&S;
6     }while(sub!=S);
7 }
8
9 void k_sub_set(int k,int n){
10     int comb=(1<<k)-1,S=1<<n;
11     while(comb<S){
12         //對大小為k的子集的處理
13         int x=comb&-comb,y=comb+x;
14         comb=((comb&y)/x>>1)|y;
15     }
16 }

```

6.3 cantor expansion

```

1 int factorial[MAXN];
2 void init(){
3     factorial[0]=1;
4     for(int i=1;i<MAXN;++i)factorial[i]=
factorial[i-1]*i;
5 }
6 int encode(const vector<int> &s){
7     int n=s.size(),res=0;
8     for(int i=0;i<n;++i){
9         int t=0;
10        for(int j=i+1;j<n;++j)
11            if(s[j]<s[i])++t;
12        res+=t*factorial[n-i-1];
13    }
14    return res;
15 }
16 vector<int> decode(int a,int n){
17     vector<int> res;
18     vector<bool> vis(n,0);
19     for(int i=n-1;i>=0;--i){
20         int t=a/factorial[i],j;
21         for(j=0;j<n;++j)
22             if(!vis[j]){
23                 if(t==0)break;
24                 --t;
25             }
26         res.push_back(j);
27         vis[j]=1;
28         a%=factorial[i];
29     }
30     return res;
31 }

```

6.4 FFT

```

1 template<typename T,typename VT=vector<
complex<T> >>
2 struct FFT{
3     const T pi;
4     FFT(const T pi=acos((T)-1)):pi(pi){}
5     unsigned bit_reverse(unsigned a,int len){
6         a=((a&0x55555555U)<<1)|((a&0xAAAAAAAAU)>>1);
7         a=((a&0x33333333U)<<2)|((a&0xCCCCCCCCU)>>2);
8         a=((a&0x0F0F0F0FU)<<4)|((a&0xFF0F0F0FU)>>4);
9         a=((a&0x00FF00FFU)<<8)|((a&0xFFFF0000U)>>8);
10        a=((a&0x0000FFFFU)<<16)|((a&0xFFFF0000U)>>16);
11        return a>>(32-len);
12    }
13    void fft(bool is_inv,VT &in,VT &out,int N)
{
14        int bitlen=__lg(N),num=is_inv?-1:1;
15        for(int i=0;i<N;++i)out[bit_reverse(i,
bitlen)]=in[i];
16        for(int step=2;step<=N;step<=1){
17            const int mh=step>>1;
18            for(int i=0;i<mh;++i){
19                complex<T> wi=exp(complex<T>(0,i*num
*pi/mh));
20                for(int j=i;j<N;j+=step){

```

```

21     int k=j+mh;
22     complex<T> u=out[j],t=wi*out[k];
23     out[j]=u+t;
24     out[k]=u-t;
25 }
26 }
27 }
28 if(is_inv)for(int i=0;i<N;++i)out[i]/=N;
29 }
30 };

```

6.5 find real root

```

1 // an*x^n + ... + a1x + a0 = 0;
2 int sign(double x){
3     return x < -eps ? -1 : x > eps;
4 }
5
6 double get(const vector<double>&coef, double
7     x){
8     double e = 1, s = 0;
9     for(auto i : coef) s += i*e, e *= x;
10    return s;
11 }
12
13 double find(const vector<double>&coef, int n
14     , double lo, double hi){
15     double sign_lo, sign_hi;
16     if( !(sign_lo = sign(get(coef,lo))) )
17         return lo;
18     if( !(sign_hi = sign(get(coef,hi))) )
19         return hi;
20     if(sign_lo * sign_hi > 0) return INF;
21     for(int stp = 0; stp < 100 && hi - lo >
22         eps; ++stp){
23         double m = (lo+hi)/2.0;
24         int sign_mid = sign(get(coef,m));
25         if(!sign_mid) return m;
26         if(sign_lo*sign_mid < 0) hi = m;
27         else lo = m;
28     }
29     return (lo+hi)/2.0;
30 }
31
32 vector<double> cal(vector<double>coef, int n
33 ){
34     vector<double>res;
35     if(n == 1){
36         if(sign(coef[1])) res.pb(-coef[0]/coef
37             [1]);
38         return res;
39     }
40     vector<double>dcoef(n);
41     for(int i = 0; i < n; ++i) dcoef[i] = coef
42         [i+1]*(i+1);
43     vector<double>droot = cal(dcoef, n-1);
44     droot.insert(droot.begin(), -INF);
45     droot.pb(INF);
46     for(int i = 0; i+1 < droot.size(); ++i){
47         double tmp = find(coef, n, droot[i],
48             droot[i+1]);
49         if(tmp < INF) res.pb(tmp);
50     }
51 }

```

6.6 FWT

```

1 vector<int> F_OR_T(vector<int> f, bool
2     inverse){
3     for(int i=0; (2<<i)<=f.size(); ++i)
4         for(int j=0; j<f.size(); j+=2<<i)
5             for(int k=0; k<(1<<i); ++k)
6                 f[j+k+(1<<i)] += f[j+k]*(inverse
7                     ?-1:1);
8     return f;
9 }
10 vector<int> rev(vector<int> A) {
11     for(int i=0; i<A.size(); i+=2)
12         swap(A[i],A[i^(A.size()-1)]);
13     return A;
14 }
15 vector<int> F_AND_T(vector<int> f, bool
16     inverse){
17     for(int i=0; (2<<i)<=f.size(); ++i)
18         for(int j=0; j<f.size(); j+=2<<i)
19             for(int k=0; k<(1<<i); ++k){
20                 int u=f[j+k], v=f[j+k+(1<<i)];
21                 f[j+k+(1<<i)] = u-v, f[j+k] = u+v;
22             }
23     if(inverse) for(auto &a:f) a/=f.size();
24     return f;
25 }

```

6.7 LinearCongruence

```

1 pair<LL,LL> LinearCongruence(LL a[],LL b[],
2     LL m[],int n) {
3     // a[i]*x = b[i] ( mod m[i] )
4     for(int i=0;i<n;++i) {
5         LL x, y, d = extgcd(a[i],m[i],x,y);
6         if(b[i]%d!=0) return make_pair(-1LL,0LL);
7         m[i] /= d;
8         b[i] = LLmul(b[i]/d,x,m[i]);
9     }
10    LL lastb = b[0], lastm = m[0];
11    for(int i=1;i<n;++i) {
12        LL x, y, d = extgcd(m[i],lastm,x,y);
13        if((lastb-b[i])%d!=0) return make_pair
14            (-1LL,0LL);
15        lastb = LLmul((lastb-b[i])/d,x,(lastm/d)
16            )*m[i];
17    }
18 }

```

```

14     lastm = (lastm/d)*m[i];
15     lastb = (lastb+b[i])%lastm;
16 }
17 return make_pair(lastb<0?lastb+lastm:lastb
18     ,lastm);
19 }

```

6.8 Lucas

```

1 ll C(ll n, ll m, ll p){// n!/m!/(n-m)!
2     if(n<m) return 0;
3     return f[n]*inv(f[m],p)%p*inv(f[n-m],p)%p;
4 }
5 ll L(ll n, ll m, ll p){
6     if(!m) return 1;
7     return C(n%p,m%p,p)*L(n/p,m/p,p)%p;
8 }
9 ll Wilson(ll n, ll p){ // n!%p
10    if(!n)return 1;
11    ll res=Wilson(n/p, p);
12    if((n/p)%2) return res*(p-f[n%p])%p;
13    return res*f[n%p]%p; //(p-1)!%p=-1
14 }

```

6.9 Matrix

```

1 template<typename T>
2 struct Matrix{
3     using rt = std::vector<T>;
4     using mt = std::vector<rt>;
5     using matrix = Matrix<T>;
6     int r,c;
7     mt m;
8     Matrix(int r,int c):r(r),c(c),m(r,rt(c)){}
9     rt& operator[](int i){return m[i];}
10    matrix operator+(const matrix &a){
11        matrix rev(r,c);
12        for(int i=0;i<r;++i)
13            for(int j=0;j<c;++j)
14                rev[i][j]=m[i][j]+a.m[i][j];
15        return rev;
16    }
17    matrix operator-(const matrix &a){
18        matrix rev(r,c);
19        for(int i=0;i<r;++i)
20            for(int j=0;j<c;++j)
21                rev[i][j]=m[i][j]-a.m[i][j];
22        return rev;
23    }
24    matrix operator*(const matrix &a){
25        matrix rev(r,a.c);
26        matrix tmp(a.c,a.r);
27        for(int i=0;i<a.r;++i)
28            for(int j=0;j<a.c;++j)
29                tmp[j][i]=a.m[i][j];
30        for(int i=0;i<r;++i)
31            for(int j=0;j<a.c;++j)
32                for(int k=0;k<c;++k)
33                    rev.m[i][j]+=m[i][k]*tmp[j][k];
34        return rev;
35    }
36 }

```

```

35 }
36 bool inverse(){
37     Matrix t(r,r+c);
38     for(int y=0;y<r;y++){
39         t.m[y][c+y] = 1;
40         for(int x=0;x<c;++x)
41             t.m[y][x]=m[y][c+x];
42     }
43     if( !t.gas() )
44         return false;
45     for(int y=0;y<r;y++){
46         for(int x=0;x<c;++x)
47             m[y][x]=t.m[y][c+x]/t.m[y][y];
48         return true;
49     }
50 }
51 T gas(){
52     vector<T> lazy(r,1);
53     bool sign=false;
54     for(int i=0;i<r;++i){
55         if( m[i][i]==0 ){
56             int j=i+1;
57             while(j<r&&!m[j][i])j++;
58             if(j==r)continue;
59             m[i].swap(m[j]);
60             sign=!sign;
61         }
62         for(int j=0;j<r;++j){
63             if(i==j)continue;
64             lazy[j]=lazy[j]*m[i][i];
65             T mx=m[j][i];
66             for(int k=0;k<c;++k)
67                 m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx;
68         }
69     }
70     T det=sign?-1:1;
71     for(int i=0;i<r;++i){
72         det = det*m[i][i];
73         det = det/lazy[i];
74         for(auto &j:m[i])j/=lazy[i];
75     }
76     return det;
77 }

```

6.10 MillerRobin

```

1 ULL LLmul(ULL a, ULL b, const ULL &mod) {
2     LL ans=0;
3     while(b) {
4         if(b&1) {
5             ans+=a;
6             if(ans>=mod) ans-=mod;
7         }
8         a<<=1, b>>=1;
9         if(a>=mod) a-=mod;
10    }
11    return ans;
12 }
13 ULL mod_mul(ULL a,ULL b,ULL m){
14     a%=m,b%=m;/* fast for m < 2^58 */
15     ULL y=(ULL)((double)a*b/m+0.5);
16     ULL r=(a*b-y*m)%m;
17 }

```

```

17 | return r<0?r+m:r;
18 | }
19 | template<typename T>
20 | T pow(T a,T b,T mod){//a^b%mod
21 |     T ans=1;
22 |     for(;b;a=mod_mul(a,a,mod),b>>=1)
23 |         if(b&1)ans=mod_mul(ans,a,mod);
24 |     return ans;
25 | }
26 | int sprp[3]={2,7,61};//int範圍可解
27 | int llsprp
    [7]={2,325,9375,28178,450775,9780504,
28 | 1795265022};//至少unsigned Long Long範圍
29 | template<typename T>
30 | bool isprime(T n,int *sprp,int num){
31 |     if(n==2)return 1;
32 |     if(n<2||n%2==0)return 0;
33 |     int t=0;
34 |     T u=n-1;
35 |     for(;u%2==0;++t)u>>=1;
36 |     for(int i=0;i<num;++i){
37 |         T a=sprp[i]%n;
38 |         if(a==0||a==1||a==n-1)continue;
39 |         T x=pow(a,u,n);
40 |         if(x==1||x==n-1)continue;
41 |         for(int j=0;j<t;++j){
42 |             x=mod_mul(x,x,n);
43 |             if(x==1)return 0;
44 |             if(x==n-1)break;
45 |         }
46 |         if(x==n-1)continue;
47 |         return 0;
48 |     }
49 |     return 1;
50 | }

```

6.11 NTT

```

1 | 2615053605667*(2^18)+1,3
2 | 15*(2^27)+1,31
3 | 479*(2^21)+1,3
4 | 7*17*(2^23)+1,3
5 | 3*3*211*(2^19)+1,5
6 | 25*(2^22)+1,3
7 | template<typename T,typename VT=vector<T>>
8 | struct NTT{
9 |     const T P,G;
10 |    NTT(T p=(1<<23)*7*17+1,T g=3):P(p),G(g){
11 |        unsigned bit_reverse(unsigned a,int len){
12 |            //Look FFT.cpp
13 |        }
14 |        T pow_mod(T n,T k,T m){
15 |            T ans=1;
16 |            for(n=(n>=m?n%m:n);k>>=1){
17 |                if(k&1)ans=ans*n%m;
18 |                n=n*n%m;
19 |            }
20 |            return ans;
21 |        }
22 |        void ntt(bool is_inv,VT &in,VT &out,int N)
23 |        {
24 |            int bitlen=__lg(N);

```

```

24 |     for(int i=0;i<N;++i)out[bit_reverse(i,
    bitlen)]=in[i];
25 |     for(int step=2,id=1;step<=N;step<=1,++
    id){
26 |         T wn=pow_mod(G,(P-1)>>id,P),wi=1,u,t;
27 |         const int mh=step>>1;
28 |         for(int i=0;i<mh;++i){
29 |             for(int j=i;j<N;j+=step){
30 |                 u=out[j],t=wi*out[j+mh]%P;
31 |                 out[j]=u+t;
32 |                 out[j+mh]=u-t;
33 |                 if(out[j]>=P)out[j]-=P;
34 |                 if(out[j+mh]<0)out[j+mh]+=P;
35 |             }
36 |             wi=wi*wn%P;
37 |         }
38 |     }
39 |     if(is_inv){
40 |         for(int i=1;i<N/2;++i)swap(out[i],out[
    N-i]);
41 |         T invn=pow_mod(N,P-2,P);
42 |         for(int i=0;i<N;++i)out[i]=out[i]*invn
    %P;
43 |     }
44 | }
45 | };

```

6.12 Simpson

```

1 | double simpson(double a,double b){
2 |     double c=a+(b-a)/2;
3 |     return (F(a)+4*F(c)+F(b))*(b-a)/6;
4 | }
5 | double asr(double a,double b,double eps,
    double A){
6 |     double c=a+(b-a)/2;
7 |     double L=simpson(a,c),R=simpson(c,b);
8 |     if( abs(L+R-A)<15*eps )
9 |         return L+R+(L+R-A)/15.0;
10 |    return asr(a,c,eps/2,L)+asr(c,b,eps/2,R);
11 | }
12 | double asr(double a,double b,double eps){
13 |     return asr(a,b,eps,simpson(a,b));
14 | }

```

6.13 外星模運算

```

1 | //a[0]^a[1]^a[2]^...
2 | #define maxn 1000000
3 | int euler[maxn+5];
4 | bool is_prime[maxn+5];
5 | void init_euler(){
6 |     is_prime[1]=1;//不是質數
7 |     for(int i=1;i<=maxn;i++)euler[i]=i;
8 |     for(int i=2;i<=maxn;i++){
9 |         if(!is_prime[i]){//是質數
10 |             euler[i]-=1;
11 |             for(int j=i<1;j<=maxn;j+=i){
12 |                 is_prime[j]=1;

```

```

13 |         euler[j]=euler[j]/i*(i-1);
14 |     }
15 | }
16 | }
17 | }
18 | LL pow(LL a,LL b,LL mod){//a^b%mod
19 |     LL ans=1;
20 |     for(;b;a=a%mod,b>>=1)
21 |         if(b&1)ans=ans*a%mod;
22 |     return ans;
23 | }
24 | bool isless(LL *a,int n,int k){
25 |     if(*a==1)return k>1;
26 |     if(--n==0)return *a<k;
27 |     int next=0;
28 |     for(LL b=1;b<k;++next)
29 |         b*=a;
30 |     return isless(a+1,n,next);
31 | }
32 | LL high_pow(LL *a,int n,LL mod){
33 |     if(*a==1||--n==0)return *a%mod;
34 |     int k=0,r=euler[mod];
35 |     for(LL tma=1;tma!=pow(*a,k+r,mod);++k)
36 |         tma=tma*(*a)%mod;
37 |     if(isless(a+1,n,k))return pow(*a,high_pow(
    a+1,n,k),mod);
38 |     int tmd=high_pow(a+1,n,r), t=(tmd-k+r)%r;
39 |     return pow(*a,k+t,mod);
40 | }
41 | LL a[1000005];
42 | int t,mod;
43 | int main(){
44 |     init_euler();
45 |     scanf("%d",&t);
46 |     #define n 4
47 |     while(t--){
48 |         for(int i=0;i<n;++i)scanf("%lld",&a[i]);
49 |         scanf("%d",&mod);
50 |         printf("%lld\n",high_pow(a,n,mod));
51 |     }
52 |     return 0;
53 | }

```

6.14 數位統計

```

1 | 11 d[65], dp[65][2];//up區間是不是完整
2 | 11 dfs(int p,bool is8,bool up){
3 |     if(!p)return 1; // 回傳0是不是答案
4 |     if(!up&&~dp[p][is8])return dp[p][is8];
5 |     int mx = up?d[p]:9;//可以用的有那些
6 |     ll ans=0;
7 |     for(int i=0;i<=mx;++i){
8 |         if( is8&&i==7 )continue;
9 |         ans += dfs(p-1,i==8,up&&i==mx);
10 |     }
11 |     if(!up)dp[p][is8]=ans;
12 |     return ans;
13 | }
14 | 11 f(ll N){
15 |     int k=0;
16 |     while(N){ // 把數字先分解到陣列
17 |         d[++k] = N%10;

```

```

18 |     N/=10;
19 | }
20 | return dfs(k,false,true);
21 | }

```

6.15 質因數分解

```

1 | LL func(const LL n,const LL mod,const int c)
    {
2 |     return (LLmul(n,n,mod)+c+mod)%mod;
3 | }
4 | }
5 | LL pollorroho(const LL n, const int c) //循環
    長度
6 |     LL a=1, b=1;
7 |     a=func(a,n,c)%n;
8 |     b=func(b,n,c)%n; b=func(b,n,c)%n;
9 |     while(gcd(abs(a-b),n)==1) {
10 |         a=func(a,n,c)%n;
11 |         b=func(b,n,c)%n; b=func(b,n,c)%n;
12 |     }
13 |     return gcd(abs(a-b),n);
14 | }
15 | void prefactor(LL &n, vector<LL> &v) {
16 |     for(int i=0;i<12;++i) {
17 |         while(n%prime[i]==0) {
18 |             v.push_back(prime[i]);
19 |             n/=prime[i];
20 |         }
21 |     }
22 | }
23 | }
24 | void smallfactor(LL n, vector<LL> &v) {
25 |     if(n<MAXPRIME) {
26 |         while(isp[(int)n]) {
27 |             v.push_back(isp[(int)n]);
28 |             n/=isp[(int)n];
29 |         }
30 |     }
31 |     v.push_back(n);
32 | } else {
33 |     for(int i=0;i<primecnt&&prime[i]*prime[i]
        <=n;++i) {
34 |         while(n%prime[i]==0) {
35 |             v.push_back(prime[i]);
36 |             n/=prime[i];
37 |         }
38 |     }
39 |     if(n!=1) v.push_back(n);
40 | }
41 | }
42 | void comfactor(const LL &n, vector<LL> &v) {
43 |     if(n<1e9) {
44 |         smallfactor(n,v);
45 |         return;
46 |     }
47 |     if(Isprime(n)) {
48 |         v.push_back(n);
49 |         return;
50 |     }
51 | }
52 | LL d;

```



```

53 for(int c=3; ++c) {
54     d = pollrho(n,c);
55     if(d!=n) break;
56 }
57 comfactor(d,v);
58 comfactor(n/d,v);
59 }
60
61 void Factor(const LL &x, vector<LL> &v) {
62     LL n = x;
63     if(n==1) { puts("Factor 1"); return; }
64     prefactor(n,v);
65     if(n==1) return;
66     comfactor(n,v);
67     sort(v.begin(),v.end());
68 }
69
70 void AllFactor(const LL &n,vector<LL> &v) {
71     vector<LL> tmp;
72     Factor(n,tmp);
73     v.clear();
74     v.push_back(1);
75     int len;
76     LL now=1;
77     for(int i=0;i<tmp.size(); ++i) {
78         if(i==0 || tmp[i]!=tmp[i-1]) {
79             len = v.size();
80             now = 1;
81         }
82         now*=tmp[i];
83         for(int j=0;j<len; ++j)
84             v.push_back(v[j]*now);
85     }
86 }

```

7 String

7.1 ac 自動機

```

1 int trie[(int)5e5 + 5][26], node_cnt = 0,
2   fail[(int)5e5 + 5]; // ac_automachine
3 int pattern_end[(int)5e5 + 5];
4 void insert(string s, int id)
5 {
6     int u = 0;
7     for(char c : s){
8         int v = c - 'a';
9         if(!trie[u][v]) trie[u][v] = ++
10            node_cnt;
11         u = trie[u][v];
12     }
13     pattern_end[id] = u;
14 }
15 void build_ac_automachine()
16 {
17     //bfs
18     queue<int> q;
19     for(int i = 0; i < 26; i++) if(trie[0][i]

```

```

20     for(int i = 0; i < 26; i++){
21         if(trie[x][i] > 0){
22             fail[trie[x][i]] = trie[fail
23                [x]][i];
24             q.push(trie[x][i]);
25         }
26         else trie[x][i] = trie[fail[x]][i];
27     }
28 }

```

7.2 hash

```

1 const int a = 31;
2 const int mod1 = 1e9 + 7;
3 const int mod2 = 1e9 + 9;
4 struct BIT{
5     int n;
6     vector<int> bit1, bit2;
7     BIT(int _n){
8         n = _n;
9         bit1.resize(n + 1, 0);
10        bit2.resize(n + 1, 0);
11    }
12    void add(int x, int val1, int val2){
13        int y = x;
14        for(; x <= n; x += x & -x) bit1[x] =
15            (bit1[x] + val1) % mod1;
16        for(; y <= n; y += y & -y) bit2[y] =
17            (bit2[y] + val2) % mod2;
18    }
19    int que1(int x){
20        int ans = 0;
21        for(; x; x -= x & -x) ans = (ans +
22            bit1[x]) % mod1;
23        return ans;
24    }
25    int que2(int x){
26        int ans = 0;
27        for(; x; x -= x & -x) ans = (ans +
28            bit2[x]) % mod2;
29        return ans;
30    }
31    int query1(int l, int r){
32        return (que1(r) - que1(l - 1) + mod1)
33            % mod1;
34    }
35    int query2(int l, int r){
36        return (que2(r) - que2(l - 1) + mod2)
37            % mod2;
38    }
39 }
40 void solve()
41 {
42     string s;
43     cin >> s;
44     int n = s.size();
45     vector<int> f1(n, 1), f2(n, 1);
46     BIT bit(n);
47     for(int i = 1; i < n; i++) f1[i] = (f1[i

```

```

48     for(int i = 1; i < n; i++) f2[i] = (f2[i
49         - 1] * a) % mod2;
50     for(int i = 0; i < n; i++) bit.add(i +
51         1, f1[i] * (s[i] - 'a' + 1) % mod1,
52         f2[i] * (s[i] - 'a' + 1) % mod2);
53     set<pii> cnt;
54     for(int i = k; i <= n; i++){
55         int sum1 = bit.query1(i - k + 1, i)
56             * fpow(f1[i - k], mod1 - 2, mod1)
57             % mod1;
58         int sum2 = bit.query2(i - k + 1, i)
59             * fpow(f2[i - k], mod2 - 2, mod2)
60             % mod2;
61         cnt.insert({sum1, sum2});
62     }
63     cout << cnt.size() << "\n";
64 }

```

7.3 KMP

```

1 vector<int> pi(s.size(), 0);
2 for(int i = 1, j = 0; i < s.size(); i++){
3     while(j > 0 and s[i] != s[j]) j = pi[j -
4         1];
5     if(s[i] == s[j]) j++;
6     pi[i] = j;
7 }

```

7.4 manacher

```

1 string s, t = "^";
2 cin >> s;
3 for(char c : s){
4     t += "#";
5     t += c;
6 }
7 t += "#";
8 vector<int> p(t.size(), 0);
9 int c = 0, r = 0, len = 0, center = 0;
10 for(int i = 1; i < t.size() - 1; i++){
11     if(i < r) p[i] = min(r - i, p[2 * c - i
12         ]);
13     while(t[i + p[i] + 1] == t[i - p[i] -
14         1]) p[i]++;
15     if(i + p[i] > r){
16         r = i + p[i];
17         c = i;
18     }
19     if(p[i] > len){
20         len = p[i];
21         center = i;
22     }
23 }
24 cout << s.substr((center - len) / 2, len) <<
25     "\n"; //Longest Palindrome

```

7.5 suffix-array

```

1 vector<int> buildSuffixArray(string s, int n
2     )
3 {
4     vector<int> sa(n), tmp(n), rank(n);
5     /*
6     sa[i]: 從sa[i]開始的後綴字串是第i小
7     rank[i]: 從i開始的後綴字串是第幾小
8     tmp: rank的暫存
9     */
10    for(int i = 0; i < n; i++){ // 初始化長
11        度為1的後綴字串
12        sa[i] = i;
13        rank[i] = s[i] - 'a';
14    }
15    for(int k = 1; k < n; k <= 1){ //比較
16        長度為2*k的字串
17        auto cmp = [&](int i, int j){ //比較
18            if(rank[i] != rank[j]) return
19                rank[i] < rank[j];
20            int ri = i + k < n ? rank[i + k]
21                : -1;
22            int rj = j + k < n ? rank[j + k]
23                : -1;
24            return ri < rj;
25        };
26        sort(sa.begin(), sa.end(), cmp); //
27        sa 按照字典序(rank)排列
28        /*
29        更新rank
30        要先用tmp存，因為cmp會用到rank的舊資
31        料
32        */
33        tmp[sa[0]] = 0;
34        for(int i = 1; i < n; i++) tmp[sa[i]
35            ] = tmp[sa[i - 1]] + (cmp[sa[i]
36                - 1, sa[i]] ? 1 : 0);
37        rank = tmp;
38    }
39    return sa;
40 }
41 vector<int> buildLCP(string s, vector<int>
42     sa, int n)
43 {
44     vector<int> rank(n, 0); // rank[i]: 從s[
45         i]開始的字尾字串按照字典序排序後的位
46         置
47     for(int i = 0; i < n; i++) rank[sa[i]] =
48         i;
49     int h = 0; // 因為是按字串s原始位置順序計
50         算，用前綴相似的特性，h每次最多只會
51         減少1
52     vector<int> lcp(n, 0); // 與字典序前一個
53         的字尾字串的最長共同前綴長度
54     for(int i = 0; i < n; i++){
55         if(rank[i] > 0){
56             int j = sa[rank[i] - 1]; // 字典
57                 序前一個的字尾字串
58             while(i + h < n and j + h < n
59                 and s[i + h] == s[j + h]) h
60                 ++;
61         }
62     }
63 }

```

```

41         lcp[rank[i]] = h;
42         if (h > 0) h--;
43     }
44 }
45 return lcp;
46 }
47 int ans = 0, n = s.size();
48 // ans: the number of distinct substrings
49 // that appear in a string
50 for(int i = 0; i < n; i++) ans += n - sa[i]
51     - lcp[i];
52 /*
53 (n - sa[i]): 這個字尾本來可以貢獻這麼多個子
54 字串。
55 lcp[i]: 扣掉跟字典序前一個字尾長得一模一樣的
56 前綴，因為那些已經算過了。
57 */

```

```

39 for(auto v:rG[u]) if(v=dfn[v]){
40     find(v,i);
41     semi[i]=min(semi[i],semi[best[v]]);
42 }
43 tree[semi[i]].push_back(i);
44 for(auto v:tree[pa[i]]){
45     find(v, pa[i]);
46     idom[v] = semi[best[v]]==pa[i]
47         ? pa[i] : best[v];
48 }
49 tree[pa[i]].clear();
50 }
51 for(int i=2; i<=dfnCnt; ++i){
52     if(idom[i] != semi[i]){
53         idom[i] = idom[idom[i]];
54         tree[id[idom[i]]].push_back(id[i]);
55     }
56 }
57 }dom;

```

```

42     addedge(a,n(b));
43     addedge(b,n(a));
44 }
45 while(M--){
46     scanf("%d%d",&a,&b);
47     a = a>0?a*2-1:-a*2;
48     b = b>0?b*2-1:-b*2;
49     // A or B
50     addedge(n(a),b);
51     addedge(n(b),a);
52 }
53 solve();
54 bool check=true;
55 for(int i=1;i<=2*N;++i)
56     if(scc[i]==scc[n(i)])
57         check=false;
58 if(check){
59     printf("%d\n",N);
60     for(int i=1;i<=2*N;i+=2){
61         if(scc[i]>scc[i+2*N]) putchar('+');
62         else putchar('-');
63     }
64     puts("");
65 }else puts("0");
66 return 0;
67 }

```

```

32     low[u]=min(low[u],vis[v]);
33 }
34 if(vis[u]==low[u]){//處理BCC
35     ++bcc_cnt;// 1-base
36     do bcc_id[v=st[--top]]=bcc_cnt;//每個點
37         所在的BCC
38     while(v!=u);
39 }
40 void bcc_init(int n){
41     Time=bcc_cnt=bridge_cnt=top=0;
42     E.clear();
43     for(int i=1;i<=n;++i){
44         G[i].clear();
45         vis[i]=bcc_id[i]=0;
46     }
47 }

```

8 Tarjan

8.1 dominator tree

```

1 struct dominator_tree{
2     static const int MAXN=5005;
3     int n;// 1-base
4     vector<int> G[MAXN], rG[MAXN];
5     int pa[MAXN], dfn[MAXN], id[MAXN], dfnCnt;
6     int semi[MAXN], idom[MAXN], best[MAXN];
7     vector<int> tree[MAXN]; // tree here
8     void init(int _n){
9         n = _n;
10        for(int i=1; i<=n; ++i)
11            G[i].clear(), rG[i].clear();
12    }
13    void add_edge(int u, int v){
14        G[u].push_back(v);
15        rG[v].push_back(u);
16    }
17    void dfs(int u){
18        id[dfn[u]]=++dfnCnt;
19        for(auto v:G[u]) if(!dfn[v])
20            dfs(v),pa[dfn[v]]=dfn[u];
21    }
22    int find(int y,int x){
23        if(y <= x) return y;
24        int tmp = find(pa[y],x);
25        if(semi[best[y]] > semi[best[pa[y]]])
26            best[y] = best[pa[y]];
27        return pa[y] = tmp;
28    }
29    void tarjan(int root){
30        dfnCnt = 0;
31        for(int i=1; i<=n; ++i){
32            dfn[i] = idom[i] = 0;
33            tree[i].clear();
34            best[i] = semi[i] = i;
35        }
36        dfs(root);
37        for(int i=dfnCnt; i>1; --i){
38            int u = id[i];

```

8.2 tnfsb017 2 sat

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAXN 8001
4 #define MAXN2 MAXN*4
5 #define n(X) ((X)+2*N)
6 vector<int> v[MAXN2], rv[MAXN2], vis_t;
7 int N,M;
8 void addedge(int s,int e){
9     v[s].push_back(e);
10    rv[e].push_back(s);
11 }
12 int scc[MAXN2];
13 bool vis[MAXN2]={false};
14 void dfs(vector<int> *uv,int n,int k=-1){
15     vis[n]=true;
16     for(int i=0;i<uv[n].size();++i)
17         if(!vis[uv[n][i]])
18             dfs(uv,uv[n][i],k);
19     if(uv==v)vis_t.push_back(n);
20     scc[n]=k;
21 }
22 void solve(){
23     for(int i=1;i<=N;++i){
24         if(!vis[i])dfs(v,i);
25         if(!vis[n(i)])dfs(v,n(i));
26     }
27     memset(vis,0,sizeof(vis));
28     int c=0;
29     for(int i=vis_t.size()-1;i>=0;--i)
30         if(!vis[vis_t[i]])
31             dfs(rv,vis_t[i],c++);
32 }
33 int main(){
34     int a,b;
35     scanf("%d%d",&N,&M);
36     for(int i=1;i<=N;++i){
37         // (A or B)&(!A & !B) A^B
38         a=i*2-1;
39         b=i*2;
40         addedge(n(a),b);
41         addedge(n(b),a);

```

8.3 橋連通分量

```

1 #define N 1005
2 struct edge{
3     int u,v;
4     bool is_bridge;
5     edge(int u=0,int v=0):u(u),v(v),is_bridge
6         (0){}
7 };
8 vector<edge> E;
9 vector<int> G[N]; // 1-base
10 int low[N],vis[N],Time;
11 int bcc_id[N],bridge_cnt,bcc_cnt; // 1-base
12 int st[N],top; // BCC用
13 void add_edge(int u,int v){
14     G[u].push_back(E.size());
15     E.emplace_back(u,v);
16     G[v].push_back(E.size());
17     E.emplace_back(v,u);
18 }
19 void dfs(int u,int re=-1){//u當前點，re為u連
20     接前一個點的邊
21     int v;
22     low[u]=vis[u]=++Time;
23     st[top++]=u;
24     for(int e:G[u]){
25         v=E[e].v;
26         if(!vis[v]){
27             dfs(v,e^1);//e^1反向邊
28             low[u]=min(low[u],low[v]);
29             if(vis[u]<low[v]){
30                 E[e].is_bridge=E[e^1].is_bridge=1;
31                 ++bridge_cnt;

```

8.4 雙連通分量 & 割點

```

1 #define N 1005
2 vector<int> G[N]; // 1-base
3 vector<int> bcc[N]; //存每塊雙連通分量的點
4 int low[N],vis[N],Time;
5 int bcc_id[N],bcc_cnt; // 1-base
6 bool is_cut[N]; //是否為割點
7 int st[N],top;
8 void dfs(int u,int pa=-1){//u當前點，pa父親
9     int t, child=0;
10    low[u]=vis[u]=++Time;
11    st[top++]=u;
12    for(int v:G[u]){
13        if(!vis[v]){
14            dfs(v,u),++child;
15            low[u]=min(low[u],low[v]);
16            if(vis[u]<=low[v]){
17                is_cut[u]=1;
18                bcc[++bcc_cnt].clear();
19                do{
20                    bcc_id[t=st[--top]]=bcc_cnt;
21                    bcc[bcc_cnt].push_back(t);
22                }while(t!=v);
23                bcc_id[u]=bcc_cnt;
24                bcc[bcc_cnt].push_back(u);
25            }
26        }else if(vis[v]<vis[u]&&v!=pa){//反向邊
27            low[u] = min(low[u],vis[v]);
28        }//u是dfs樹的根要特判
29        if(pa!=-1&&child<2)is_cut[u]=0;
30    }
31    void bcc_init(int n){
32        Time=bcc_cnt=top=0;
33        for(int i=1;i<=n;++i){
34            G[i].clear();
35            is_cut[i]=vis[i]=bcc_id[i]=0;
36        }
37    }

```

9 Tree Problem

9.1 find-centroid

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define KCC ios::sync_with_stdio(0);cin.tie
5 (0);
6 #define pb push_back
7
8 int subtree[(int)2e5 + 5], n;
9 vector<int> g[(int)2e5 + 5];
10 void dfs(int now, int par)
11 {
12     subtree[now] = 1;
13     for(auto w : g[now]){
14         if(w != par){
15             dfs(w, now);
16             subtree[now] += subtree[w];
17         }
18     }
19     return ;
20 }
21 int get(int now, int par)
22 {
23     for(auto w : g[now]){
24         if(w != par and subtree[w] * 2 > n)
25             return get(w, now);
26     }
27     return now;
28 }
29 signed main()
30 {
31     KCC
32     cin >> n;
33     for(int i = 0; i < n - 1; i++){
34         int a, b; cin >> a >> b;
35         g[a].pb(b);
36         g[b].pb(a);
37     }
38     dfs(1, 0);
39     cout << get(1, 0) << "\n";
40     return 0;
41 }

```

9.2 hld

```

1 vector<int> edge[MAXN];
2 int ar[MAXN], sz[MAXN], par[MAXN], son[MAXN], dep[MAXN], dfn[MAXN], top[MAXN], t =
3 0;
4 void dfs_son(int x, int p, int d)
5 {
6     sz[x] = 1;
7     dep[x] = d;
8     son[x] = 0;
9     for(int i : edge[x]){
10         if(i != p){
11             dfs_son(i, x, d + 1);
12         }
13     }
14     sz[x] += sz[i];
15     par[i] = x;
16     if(sz[i] > sz[son[x]]) son[x] = i;
17 }
18 void dfs_hld(int x, int p)
19 {
20     dfn[x] = ++t;
21     if(son[x] != 0){
22         top[son[x]] = top[x];
23         dfs_hld(son[x], x);
24     }
25     for(int i : edge[x]){
26         if(i != p and i != son[x]){
27             top[i] = i;
28             dfs_hld(i, x);
29         }
30     }
31 }
32 int main()
33 {
34     int n, q;
35     cin >> n >> q;
36     for(int i = 1; i <= n; i++) cin >> ar[i];
37     for(int i = 0; i < n - 1; i++){
38         int u, v;
39         cin >> u >> v;
40         edge[u].pb(v);
41         edge[v].pb(u);
42     }
43     dfs_son(1, 0, 1);
44     top[1] = 1;
45     dfs_hld(1, 0);
46
47     vector<int> v(n + 5);
48     for(int i = 1; i <= n; i++) v[dfn[i]] = ar[i];
49     seg_tree seg(n);
50     seg.build(v, 1, n, 1);
51     while(q--){
52         int op; cin >> op;
53         if(op == 1){ //change the value of
54             node s to x
55             int s, x;
56             cin >> s >> x;
57             seg.upd(dfn[s], x, 1, n, 1);
58         }
59         else{ //find the maximum value on
60             the path between nodes a and b.
61             int a, b, ans = 0;
62             cin >> a >> b;
63             while(top[a] != top[b]){
64                 if(dfn[top[a]] < dfn[top[b]]) swap(a, b);
65                 ans = max(ans, seg.query(dfn[top[a]], dfn[a], 1, n, 1));
66                 a = par[top[a]];
67             }
68             if(dfn[a] > dfn[b]) swap(a, b);
69             ans = max(ans, seg.query(dfn[a], dfn[b], 1, n, 1));
70             cout << ans << " ";
71         }
72     }
73 }

```

9.3 lca

```

1 void dfs(int now, int par)
2 {
3     deep[now] = deep[par] + 1;
4     p[now][0] = par;
5     for(int i : v[now]) if(i != par) dfs(i, now);
6     return;
7 }
8 int lca(int u, int v)
9 {
10     int x = deep[u], y = deep[v];
11     if(x > y){
12         swap(u, v);
13         swap(x, y);
14     }
15     y -= x; // x < y
16     for(int i = 25; i >= 0; i--) if(y & (1 << i)) v = p[v][i];
17     if(u == v) return u;
18     for(int i = 25; i >= 0; i--){
19         if(p[u][i] != p[v][i]){
20             u = p[u][i];
21             v = p[v][i];
22         }
23     }
24     return p[u][0];
25 }
26 for(int j = 1; j < 30; j++){
27     for(int i = 2; i <= n; i++){
28         p[i][j] = p[p[i][j - 1]][j - 1];
29     }
30 }

```

10 compare

10.1 check

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     // 1. 在迴圈外編譯，並檢查是否成功
5     cout << "Compiling files..." << endl;
6
7     // system() 回傳 0 代表成功執行
8     int c1 = system("g++ -Wall -std=c++20 sol.cpp -o sol.exe");
9     int c2 = system("g++ -Wall -std=c++20 bf.cpp -o bf.exe");
10    int c3 = system("g++ -Wall -std=c++20 gen.cpp -o gen.exe");
11 }

```

```

12 if (c1 != 0 || c2 != 0 || c3 != 0) {
13     cout << "Compilation Failed! Check if sol.cpp, bf.cpp, and gen.cpp exist." << endl;
14     return 1;
15 }
16
17 cout << "Compilation successful. Starting stress test..." << endl;
18
19 for (int T = 1; ; T++) {
20     // 2. 執行程序
21     system("gen.exe > input.txt");
22     system("sol.exe < input.txt > sol.out");
23     system("bf.exe < input.txt > bf.out");
24
25     // 3. 使用 fc /W 比對
26     if (system("fc sol.out bf.out > nul")) {
27         system("fc sol.out bf.out");
28         cout << "WA on Test #" << T << "!" << endl;
29         // 發生錯誤時停止，此時 input.txt 就是出錯的測資
30         break;
31     } else {
32         if (T % 10 == 0) cout << "Passed" << T << " tests..." << endl;
33     }
34 }
35 return 0;
36 }

```

10.2 gen

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 // 使用隨機種子，確保每次生成的數據都不同
5 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
6
7 // 生成 [l, r] 範圍內的隨機整數
8 long long rnd(long long l, long long r) {
9     return uniform_int_distribution<long long>(l, r)(rng);
10 }
11
12 // 生成一棵隨機樹 (N 個點 · N-1 條邊)
13 void gen_tree(int n) {
14     for (int i = 2; i <= n; i++) {
15         // 讓每個點 i 連向 [1, i-1] 中的隨機點
16         int u = i;
17         int v = rnd(1, i - 1);
18         cout << u << " " << v << "\n";
19     }
20 }

```

11 default

11.1 debug

```
1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 template<typename I> void _DO(I&&x){cerr<<x
9     <<endl;}
10 template<typename I, typename...T> void _DO(I
11     &&x, T&&...tail){cerr<<x<<" "; _DO(tail
12     ...);}
13 #else
14 #define dbg(...)
15 #endif
```

11.2 ext

```
1 #include<bits/extc++.h>
2 #include<ext/pd_ds/assoc_container.hpp>
3 #include<ext/pd_ds/tree_policy.hpp>
4 using namespace __gnu_cxx;
5 using namespace __gnu_pbds;
6 template<typename T>
7 using pbds_set = tree<T, null_type, less<T>,
8     rb_tree_tag,
9     tree_order_statistics_node_update>;
10 template<typename T, typename U>
11 using pbds_map = tree<T, U, less<T>,
12     rb_tree_tag,
13     tree_order_statistics_node_update>;
14 using heap=__gnu_pbds::priority_queue<int>;
15 //s.find_by_order(1); //0 base
16 //s.order_of_key(1);
```

11.3 IncStack

```
1 //Magic
2 #pragma GCC optimize "Ofast"
3 //stack resize, change esp to rsp if 64-bit
4 //system
5 asm("mov %0, %%esp\n" :: "g"(mem+10000000));
6 #pragma comment(linker, "/STACK
7 :1024000000,1024000000")
8 //linux stack resize
9 #include<sys/resource.h>
10 void increase_stack(){
11     const rlim_t ks=64*1024*1024;
12     struct rlimit rl;
13     int res=getrlimit(RLIMIT_STACK, &rl);
14     if(!res&&rl.rlim_cur<ks){
15         rl.rlim_cur=ks;
16     }
17 }
```

```
15 res=setrlimit(RLIMIT_STACK, &rl);
16 }
17 }
```

11.4 input

```
1 inline int read(){
2     int x=0; bool f=0; char c=getchar();
3     while(ch<'0' || '9'<ch)f|=ch=='-', ch=getchar
4     ();
5     while('0'<=ch&&ch<='9')x=x*10-'0'+ch, ch=
6     getchar();
7     return f?-x:x;
8 }
9 // #!/bin/bash
10 // g++ -std=c++11 -O2 -Wall -Wextra -Wno-
11 // unused-result -DDEBUG $1 && ./a.out
12 // -fsanitize=address -fsanitize=undefined
13 // -fsanitize=return
```

12 other

12.1 WhatDay

```
1 int whatday(int y, int m, int d){
2     if(m<=2)m+=12, --y;
3     if(y<1752 || y==1752&&m<9 || y==1752&&m==9&&d
4     <3)
5     return (d+2*m+3*(m+1)/5+y+y/4+5)%7;
6     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)
7     %7;
8 }
```

12.2 上下最大正方形

```
1 void solve(int n, int a[], int b[]){ // 1-base
2     int ans=0;
3     deque<int> da, db;
4     for(int l=1, r=1; r<=n; ++r){
5         while(da.size()&&a[da.back()]>a[r]){
6             da.pop_back();
7         }
8         da.push_back(r);
9         while(db.size()&&b[db.back()]>b[r]){
10             db.pop_back();
11         }
12         db.push_back(r);
13         for(int d=a[da.front()+b[db.front()]]; r-
14             l+1>d; ++l){
15             if(da.front()==l)da.pop_front();
16             if(db.front()==l)db.pop_front();
17             if(da.size()&&db.size()){
18                 d=a[da.front()+b[db.front()]];
19             }
20         }
21     }
```

```
20 ans=max(ans, r-l+1);
21 }
22 printf("%d\n", ans);
23 }
```

12.3 最大矩形

```
1 LL max_rectangle(vector<int> s){
2     stack<pair<int, int>> st;
3     st.push(make_pair(-1, 0));
4     s.push_back(0);
5     LL ans=0;
6     for(size_t i=0; i<s.size(); ++i){
7         int h=s[i];
8         pair<int, int> now=make_pair(h, i);
9         while(h<st.top().first){
10             now=st.top();
11             st.pop();
12             ans=max(ans, (LL)(i-now.second)*now.
13                 first);
14         }
15         if(h>st.top().first){
16             st.push(make_pair(h, now.second));
17         }
18     }
19     return ans;
20 }
```

13 other language

13.1 java

13.1.1 文件操作

```
1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7
8     public static void main(String args[]){
9         throws FileNotFoundException,
10         IOException
11         Scanner sc = new Scanner(new FileReader(
12             "a.in"));
13         PrintWriter pw = new PrintWriter(new
14             FileWriter("a.out"));
15         int n, m;
16         n=sc.nextInt(); //读入下一个INT
17         m=sc.nextInt();
18
19         for(ci=1; ci<=c; ++ci){
20             pw.println("Case #"+ci+": easy for
21                 output");
22         }
23     }
```

```
19 pw.close(); //关闭流并释放，这个很重要，
20 否则是没有输出的
21 sc.close(); //关闭流并释放
22 }
```

13.1.2 优先队列

```
1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3         public int compare( Point a, Point b ){
4             if( a.x < b.x || a.x == b.x && a.y < b.y )
5                 return -1;
6             else if( a.x == b.x && a.y == b.y )
7                 return 0;
8             else return 1;
9         }};
```

13.1.3 Map

```
1 Map map = new HashMap();
2 map.put("sa", "dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }
```

13.1.4 sort

```
1 static class cmp implements Comparator{
2     public int compare(Object o1, Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8
9 public static void main(String[] args)
10     throws IOException{
11     Scanner cin = new Scanner(System.in);
12     int n;
13     n=cin.nextInt();
14     BigInteger[] seg = new BigInteger[n];
15     for (int i=0; i<n; i++)
16         seg[i]=cin.nextBigInteger();
17     Arrays.sort(seg, new cmp());
18 }
```

13.2 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

13.3 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

13.4 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB "{}{}"'.format('Jin', 'KeLa'))
10 # AAA Jin BBB "KeLa!"
11
12 hello = 'hello, world\n'
13 hellos = repr(hello)
14 print(hellos) # 'hello, world\n'
15
16 x = 32.5
17 y = 40000
18 print(repr((x, y, ('spam', 'eggs'))))
19 # "(32.5, 40000, ('spam', 'eggs'))"
20
21 x = 7
22 print(eval('3 * x')) # 21

```

14 zformula

14.1 formula

14.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

14.1.2 圖論

- 對於平面圖， $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖， $E \leq 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $C_v(G)$ ，最小邊覆蓋設為 $C_e(G)$ 。對於任意連通圖：
 - $I(G) + C_v(G) = |V|$
 - $M(G) + C_e(G) = |V|$
- 對於連通二分圖：
 - $I(G) = C_v(G)$
 - $M(G) = C_e(G)$
- 最大權閉合圖：
 - $C(u, v) = \infty, (u, v) \in E$
 - $C(S, v) = W_v, W_v > 0$
 - $C(v, T) = -W_v, W_v < 0$
 - $\text{ans} = \sum_{W_v > 0} W_v - \text{flow}(S, T)$
- 最大密度子圖：
 - 求 $\max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$
 - $U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$
 - $C(u, v) = W_{(u,v)}, (u, v) \in E$ ，雙向邊
 - $C(S, v) = U, v \in V$
 - $D_u = \sum_{(u,v) \in E} W_{(u,v)}$
 - $C(v, T) = U + 2g - D_v - 2W_v, v \in V$
 - 二分搜 g ：
 $l = 0, r = U, \text{eps} = 1/n^2$
 $\text{if}((U \times |V| - \text{flow}(S, T))/2 > 0) \text{ } l = \text{mid}$
 $\text{else } r = \text{mid}$
 - $\text{ans} = \min_cut(S, T)$
 - $|E| = 0$ 要特殊判斷
- 弦圖：
 - 點數大於 3 的環都要有一條弦
 - 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
 - 最大團大小 = 色數
 - 最大獨立集：完美消除序列從前往後能選就選
 - 最小團覆蓋：最大獨立集的點和他延伸的邊構成
 - 區間圖是弦圖
 - 區間圖的完美消除序列：將區間按造又端點由小到大排序
 - 區間圖染色：用線段樹做

14.1.3 dinic 特殊圖複雜度

- 單位流： $O \left(\min \left(V^{3/2}, E^{1/2} \right) E \right)$
- 二分圖： $O \left(V^{1/2} E \right)$

14.1.4 0-1 分數規劃

$x_i \in \{0, 1\}$ ， x_i 可能會有其他限制，求 $\max \left(\frac{\sum B_i x_i}{\sum C_i x_i} \right)$

- $D(i, g) = B_i - g \times C_i$
- $f(g) = \sum D(i, g) x_i$
- $f(g) = 0$ 時 g 為最佳解， $f(g) < 0$ 沒有意義
- 因為 $f(g)$ 單調可以二分搜 g
- 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2     while(r-l>eps){
3         g=(l+r)/2;
4         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
5         找出一組合法x[i]使f(g)最大;
6         if(f(g)>0) l=g;
7         else r=g;
8     }
9     Ans = r;
10 }
11 Dinkelbach(){
12     g=任意狀態 (通常設為0);
13     do{
14         Ans=g;
15         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
16         找出一組合法x[i]使f(g)最大;
17         p=0,q=0;
18         for(i:所有元素)
19             if(x[i])p+=B[i],q+=C[i];
20         g=p/q; //更新解，注意q=0的情況
21     }while(abs(Ans-g)>EPS);
22     return Ans;
23 }

```

14.1.5 學長公式

- $\sum_{d|n} \phi(n) = n$
- $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 格雷碼 $= n \oplus (n >> 1)$
- $SG(A + B) = SG(A) \oplus SG(B)$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

14.1.6 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = n \sum_{d|n} d \times \phi(d)$

14.1.7 排組公式

- k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
- $H(n, m) \cong x_1 + x_2 \dots + x_n = k, \text{num} = C_k^{n+k-1}$
- Stirling number of $2^{nd}, n$ 人分 k 組方法數目
 - $S(0, 0) = S(n, n) = 1$
 - $S(n, 0) = 0$
 - $S(n, k) = kS(n-1, k) + S(n-1, k-1)$
- Bell number, n 人分任意多組方法數目
 - $B_0 = 1$
 - $B_n = \sum_{i=0}^n S(n, i)$
 - $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
 - $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
 - $B_p^{m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
 - From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$
- Derangement, 錯排，沒有人在自己位置上
 - $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
 - $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
 - From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

6. Binomial Equality

- $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{l+s}$
- $\sum_k \binom{m+k}{l} \binom{s}{n-k} = \binom{l+s}{l-m+n}$
- $\sum_k \binom{m+k}{m} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m-1}{l-k} \binom{q+k}{n}$
- $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- $\sum_{k \leq n} \binom{r+k}{n} = \binom{r+n+1}{m+1}$
- $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- $\sum_{k \leq m} \binom{m+r}{k} x^k y^k = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

14.1.8 冪次, 冪次和

- $a^{b \% p} = a^{b \% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
- $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
- $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
- $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
- $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
- $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
- $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
- 除了 $B_1 = -1/2$ ，剩下的奇數項都是 0
- $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

14.1.9 Burnside’s lemma

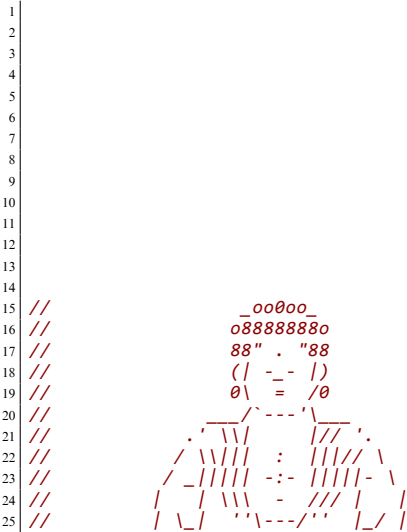
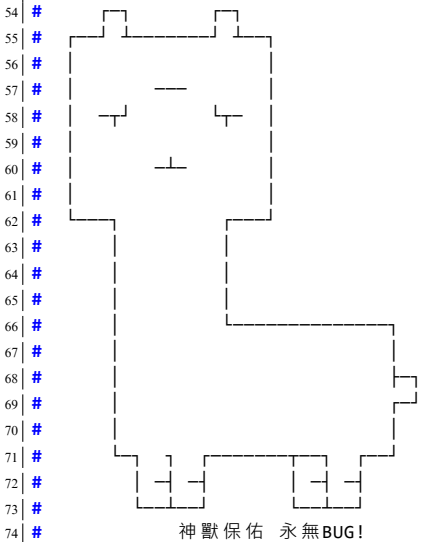
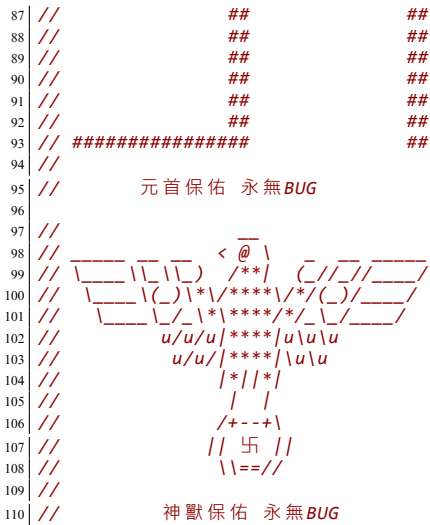
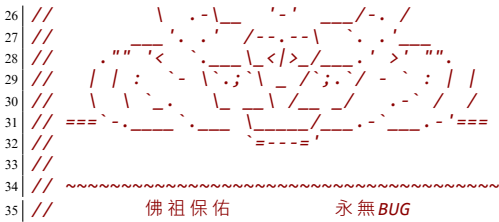
- 1. $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
- 2. $X^g = t^{c(g)}$
- 3. G 表示有幾種轉法 · X^g 表示在那種轉法下 · 有幾種是會保持對稱的 · t 是顏色數 · $c(g)$ 是循環節不動的面數。
- 4. 正立方體塗三顏色 · 轉 0 有 3^6 個元素不變 · 轉 90 有 6 種 · 每種有 3^3 不變 · 180 有 3×3^4 · 120(角) 有 8×3^2 · 180(邊) 有 6×3^3 · 全部 $\frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = \frac{57}{1}$

14.1.10 Count on a tree

- 1. Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
- 2. Unrooted tree:
 - (a) Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
 - (b) Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
- 3. Spanning Tree
 - (a) 完全圖 $n^n - 2$
 - (b) 一般圖 (Kirchhoff’s theorem) $M[i][i] = degree(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, ans = det(A)$

15 Интернационал

15.1 保佑



ACM ICPC Team Reference - CCU-88boy

Contents

1 Computational Geometry	1	2.7 動態開點線段樹	4	6.13 外星模運算	8	12 other	12
1.1 basic	1	3 Flow	4	6.14 數位統計	8	12.1 WhatDay	12
1.2 CHT	1	3.1 dinic	4	6.15 質因數分解	8	12.2 上下最大正方形	12
1.3 convex-hull	1	3.2 Edmonds-Karp	4	7 String	9	12.3 最大矩形	12
1.4 Line Segment Intersection . .	1	3.3 Min Cost Max-Flow	4	7.1 ac 自動機	9	13 other language	12
1.5 Point in Polygon	1	4 Graph	5	7.2 hash	9	13.1 java	12
1.6 半平面交集	1	4.1 bellman-ford	5	7.3 KMP	9	13.1.1 文件操作	12
1.7 最近點對	2	4.2 dijk	5	7.4 manacher	9	13.1.2 優先隊列	12
1.8 李超線段樹	2	4.3 Euler-Path	5	7.5 suffix-array	9	13.1.3 Map	12
1.9 矩形覆蓋面積	2	4.4 SCC	5	8 Tarjan	10	13.1.4 sort	12
2 Data Structure	2	4.5 tarjan scc	5	8.1 dominator tree	10	13.2 python heap	12
2.1 DSU-rollback	2	5 Linear Programming	5	8.2 tnfsb017 2 sat	10	13.3 python input	13
2.2 FHQ-treap	2	5.1 simplex	5	8.3 橋連通分量	10	13.4 python output	13
2.3 persistent-segment-tree	3	6 Number Theory	5	8.4 雙連通分量 & 割點	10	14 zformula	13
2.4 segment tree lazytag	3	6.1 basic	5	9 Tree Problem	11	14.1 formula	13
2.5 trie	3	6.2 bit set	6	9.1 find-centroid	11	14.1.1 Pick 公式	13
2.6 Weighted DSU	3	6.3 cantor expansion	6	9.2 hld	11	14.1.2 圖論	13
		6.4 FFT	6	9.3 lca	11	14.1.3 dinic 特殊圖複雜度 . .	13
		6.5 find real root	7	10 compare	11	14.1.4 0-1 分數規劃	13
		6.6 FWT	7	10.1 check	11	14.1.5 學長公式	13
		6.7 LinearCongruence	7	10.2 gen	11	14.1.6 基本數論	13
		6.8 Lucas	7	11 default	12	14.1.7 排組公式	13
		6.9 Matrix	7	11.1 debug	12	14.1.8 冪次, 冪次和	13
		6.10 MillerRobin	7	11.2 ext	12	14.1.9 Burnside's lemma . . .	14
		6.11 NTT	8	11.3 IncStack	12	14.1.10 Count on a tree	14
		6.12 Simpson	8	11.4 input	12	15 Интернационал	14
						15.1 保佑	14

ACM ICPC Judge Test - CCU-88boy

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```